

POLITECHNIKA WARSZAWSKA

WYDZIAŁ ELEKTRONIKI I TECHNIK INFORMACYJNYCH

PROGRAMOWANIE RÓWNOLEGŁE I ROZPROSZONE

Sprawozdanie - projekt 1

Autorzy:

Mateusz CHMURZYŃSKI

Filip KIERNOZEK

Bartłomiej KRAJEWSKI

Bartosz SĘDZIKOWSKI

8 grudnia 2025

Spis treści

1	Wprowadzenie	2
1.1	Etapy realizacji projektu	2
2	Metoda optymalizacji: Particle Swarm Optimization (PSO)	2
2.1	Zasada działania algorytmu	2
2.2	Implementacja sekwencyjna a równoległa	3
3	Zadania optymalizacji	3
3.1	Zadanie 1: Funkcja Schwefel	3
3.2	Zadanie 2: Funkcja Rosenbrock	3
4	Kryteria zatrzymania	4
5	Parametry eksperymentu	4
6	Wyniki eksperymentów	4
6.1	Zadanie 1: Funkcja Schwefel	4
6.2	Zadanie 2: Funkcja Rosenbrock	5
7	Wnioski i analiza	5
7.1	Analiza wydajności i przyspieszenia	5
7.2	Zbieżność algorytmu w wysokich wymiarach	5
8	Wizualizacja zbieżności algorytmu (n=2)	6
8.1	Funkcja Schwefela	6
8.2	Funkcja Rosenbrocka	6

1 Wprowadzenie

Niniejsze sprawozdanie dotyczy pierwszego etapu projektu z przedmiotu Programowanie równoległe i rozproszone. Celem projektu jest implementacja sekwencyjnej i równoległej wersji algorytmu optymalizacji, a następnie ocena przyspieszenia obliczeń w zależności od stopnia zrównoleglenia.

1.1 Etapy realizacji projektu

W ramach pierwszego etapu projektu zrealizowano następujące zadania:

1. Implementacja algorytmu Particle Swarm Optimization (PSO) w wersji sekwencyjnej).
2. Implementacja algorytmu PSO w wersji równoległej na maszyny z pamięcią wspólną z wykorzystaniem modułu `multiprocessing` w Pythonie.
3. Przeprowadzenie eksperymentów dla obu wersji algorytmu i funkcji celu dla wymiaru $n = 2$, dwóch kryteriów stopu oraz różnych rozmiarów roju (*swarm size*).
4. Analiza wydajności i ocena przyspieszenia obliczeń w zależności od rozmiaru problemu.

2 Metoda optymalizacji: Particle Swarm Optimization (PSO)

Algorytm **Particle Swarm Optimization (PSO)** jest ewolucyjną techniką optymalizacji, inspirowaną zachowaniem stad ptaków lub ławic ryb. Jest to metoda heurystyczna, wykorzystywana do znajdowania globalnych minimów funkcji celu w przestrzeni wielowymiarowej.

2.1 Zasada działania algorytmu

W algorytmie PSO, populacja potencjalnych rozwiązań, zwana **rojem (swarm)**, składa się z pojedynczych cząstek (**particles**). Każda cząstka w roju porusza się w przestrzeni poszukiwań z określoną prędkością, dostosowując swoją trajektorię na podstawie własnego najlepszego doświadczenia oraz najlepszego doświadczenia całego roju.

Każda cząstka i charakteryzuje się:

- **Pozycją** $x_i(t)$ w przestrzeni poszukiwań, gdzie t to numer iteracji.
- **Prędkością** $v_i(t)$.
- **Najlepszą dotychczasową pozycją osobistą** $pbest_i$, która dała najniższą wartość funkcji celu.

Dodatkowo, algorytm śledzi **najlepszą globalną pozycję** $gbest$, która jest najlepszą pozycją spośród wszystkich $pbest_i$ w roju.

W każdej iteracji, prędkość i pozycja cząstki są aktualizowane według następujących równań:

$$v_i(t+1) = wv_i(t) + c_1r_1(pbest_i - x_i(t)) + c_2r_2(gbest - x_i(t)) \quad (1)$$

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (2)$$

gdzie:

- w – **współczynnik bezwładności**, kontroluje wpływ poprzedniej prędkości.
- c_1 – **współczynnik kognitywny**, dążenie do własnego $pbest$.
- c_2 – **współczynnik społeczny**, dążenie do globalnego $gbest$.
- r_1, r_2 – losowe liczby z zakresu $[0, 1]$.

2.2 Implementacja sekwencyjna a równoległa

Główna różnica między implementacją sekwencyjną a równoległą dotyczy sposobu obliczania wartości funkcji celu dla całej populacji.

- **Wersja sekwencyjna:** W pliku `sequential/main.py` używana jest funkcja `np.apply_along_axis`, która przechodzi po wszystkich cząstkach w jednym wątku i oblicza ich wartości funkcji.
- **Wersja równoległa:** W pliku `parallel_processes/main.py` zastosowano podejście oparte na `multiprocessing`, które pozwala podzielić obciążenie obliczeniowe między wiele rdzeni.
 1. Populacja cząstek jest dzielona na mniejsze części (*chunks*) przez funkcję `chunkify`.
 2. Każda część trafia do osobnego procesu w puli (`Pool`).
 3. Funkcja `eval_block` jest wykonywana równoległe na wielu rdzeniach.
 4. Otrzymane wyniki są następnie zbierane i łączone w procesie głównym.

3 Zadania optymalizacji

W projekcie wykorzystano dwie funkcje testowe zaimplementowane w pliku `benchmark_functions.py`.

3.1 Zadanie 1: Funkcja Schwefel

Funkcja Schwefela jest wielomodalna, z wieloma lokalnymi minimami, co sprawia, że jest trudna dla algorytmów optymalizacji.

$$f(\mathbf{x}) = 418.9829n - \sum_{i=1}^n \left[x_i \sin(\sqrt{|x_i|}) \right]$$

Ograniczenia: $-100 \leq x_i \leq 500$. Optimum globalne $f_{min} = 0$ znajduje się w punkcie $x_i \approx 420.9687$.

3.2 Zadanie 2: Funkcja Rosenbrock

Funkcja Rosenbrocka, znana jako "dolina", posiada minimum w płaskim, wąskim parabolicznym obszarze.

$$f(\mathbf{x}) = \sum_{i=1}^{n-1} [100(x_{i+1} - x_i^2)^2 + (1 - x_i)^2]$$

Ograniczenia: $-10 \leq x_i \leq 10$. Optimum globalne $f_{min} = 0$ znajduje się w punkcie $x_i = 1$.

4 Kryteria zatrzymania

Zaimplementowano dwa warunki stopu w pliku `stop_conditions.py`:

1. **Znane optimum:** Algorytm kończy działanie, gdy norma różnicy między najlepszą znaną pozycją a znanym optimum jest mniejsza od ε_{opt} ($\|g_{best} - x_{opt}\| \leq \varepsilon$).
2. **Brak poprawy:** Algorytm kończy działanie, gdy przez m iteracji zmiana wartości funkcji celu jest mniejsza niż $\varepsilon_{no_improve}$.

5 Parametry eksperymentu

Wszystkie testy wydajnościowe przeprowadzono przy użyciu jednolitych parametrów dla obu funkcji celu oraz wszystkich wymiarów przestrzeni. Pozwoliło to na rzetelne porównanie czasów wykonania.

Parametry algorytmu PSO:

- **Rozmiar roju (`swarm_size`):** 1000 cząstek.
- **Maksymalna liczba iteracji:** 10000.
- **Współczynniki:** $w = 0.7$ (bezwładność), $c_1 = 1.5$ (komponent kognitywny), $c_2 = 1.5$ (komponent społeczny).

Kryteria zatrzymania:

1. **Kryterium 1 (Znane optimum):** Stop, gdy $\|g_{best} - x_{opt}\| \leq 10^{-3}$.
2. **Kryterium 2 (Brak poprawy):** Stop, gdy brak poprawy o 10^{-6} przez 50 iteracji.

6 Wyniki eksperymentów

Poniżej przedstawiono wyniki dla wymiarów $n = 10, 50, 100$ uzyskane na maszynie z 6-rdzeniowym procesorem. Tabela zawiera czasy wykonania dla wersji sekwencyjnej (T_{seq}) i równoległej (T_{par}) oraz obliczone przyspieszenie $S = T_{seq}/T_{par}$.

6.1 Zadanie 1: Funkcja Schwefel

Tabela 1: Wyniki dla funkcji Schwefela (Swarm Size = 1000)

Wymiar (n)	Kryterium	Iteracje	Najlepsze $f(x)$	T_{seq} [s]	T_{par} [s]	Przyspieszenie (S)
10	Znane Opt.	120	$1.27 \cdot 10^{-4}$	0.96 s	0.75 s	1.28
10	Brak Popr.	161	$1.27 \cdot 10^{-4}$	1.22 s	0.86 s	1.42
50	Znane Opt.*	10000	6726.75	90.95 s	47.96 s	1.90
50	Brak Popr.	357	6726.75	3.61 s	2.87 s	1.26
100	Znane Opt.*	10000	11854.13	114.80 s	67.20 s	1.71
100	Brak Popr.	1024	11854.13	13.06 s	10.48 s	1.25

*Algorytm osiągnął limit iteracji nie znajdując optimum z zadaną dokładnością.

6.2 Zadanie 2: Funkcja Rosenbrock

Tabela 2: Wyniki dla funkcji Rosenbrocka (Swarm Size = 1000)

Wymiar (n)	Kryterium	Iteracje	Najlepsze $f(x)$	T_{seq} [s]	T_{par} [s]	Przyspieszenie (S)
10	Znane Opt.*	10000	5.42	99.16 s	39.12 s	2.53
10	Brak Popr.	191	5.42	1.99 s	1.03 s	1.93
50	Znane Opt.*	10000	115.20	110.38 s	54.96 s	2.01
50	Brak Popr.	9003	115.21	101.85 s	54.14 s	1.88
100	Znane Opt.*	10000	119.96	138.10 s	97.86 s	1.41
100	Brak Popr.*	10000	119.96	150.53 s	106.02 s	1.42

*Algorytm osiągnął limit iteracji.

7 Wnioski i analiza

7.1 Analiza wydajności i przyspieszenia

W przeprowadzonych eksperymentach wersja równoległa algorytmu PSO okazała się **zawsze szybsza** od wersji sekwencyjnej (wartość $S > 1$ we wszystkich przypadkach).

- **Maksymalne przyspieszenie:** Najwyższą wartość $S \approx 2.53$ odnotowano dla funkcji Rosenbrocka przy $n = 10$ (Kryterium 1). Oznacza to ponad 2.5-krotne skrócenie czasu obliczeń.
- **Skalowalność:** Mimo wykorzystania 6 procesów, przyspieszenie nie osiąga wartości bliskiej 6. Jest to spowodowane prawem Amdahla – zrównoleglona jest jedynie ewaluacja funkcji celu. Aktualizacja pozycji i prędkości oraz synchronizacja procesów (w tym kosztowna serializacja danych przy przesyłaniu dużych tablic między procesami) pozostają sekwencyjne lub generują narzut.
- **Wpływ wymiarowości:** Przy dużych wymiarach ($n = 100$) przyspieszenie jest zauważalne (ok. 1.2x - 1.7x), ale ograniczone przez zwiększony koszt przesyłania większych wektorów danych do procesów podrzędnych.

7.2 Zbieżność algorytmu w wysokich wymiarach

Wyniki pokazują trudność optymalizacji w przestrzeniach wielowymiarowych ("przekleństwo wymiarowości"):

- Dla $n = 10$ algorytm radzi sobie dobrze, szczególnie dla funkcji Schwefela, gdzie znajduje optimum bliskie 0 ($1.27 \cdot 10^{-4}$).
- Dla $n = 50$ i $n = 100$ algorytm często utyka w minimach lokalnych. Dla funkcji Schwefela wartości $f(x)$ rzędu 6000-11000 świadczą o tym, że rój nie zdołał znaleźć globalnego minimum w tak dużej przestrzeni przy założonej liczbie iteracji. W takich przypadkach algorytm przerywał pracę po osiągnięciu limitu iteracji lub z powodu braku poprawy, co drastycznie wpływało na czas wykonania (np. 150 sekund dla Rosenbrocka sekwencyjnie przy $n = 100$).

Podsumowując, implementacja równoległa przynosi wymierne korzyści czasowe, szczególnie gdy algorytm musi wykonać dużą liczbę iteracji (np. przy trudnych problemach optymalizacyjnych, gdzie nie następuje szybka zbieżność).

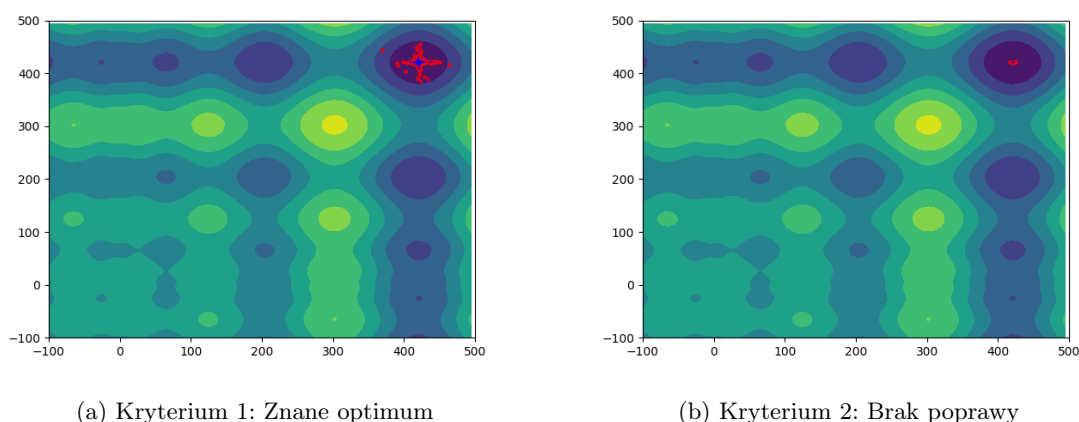
8 Wizualizacja zbieżności algorytmu ($n=2$)

W celu zobrazowania działania algorytmu PSO oraz dynamiki poruszania się roju, przeprowadzono symulację dla przestrzeni dwuwymiarowej ($n = 2$). Pozwoliło to na graficzne przedstawienie pozycji cząstek na płaszczyźnie wyznaczonej przez granice funkcji celu.

Poniższe rysunki przedstawiają **ostatnią iterację** algorytmu, czyli stan końcowy, w którym proces został zatrzymany przez jedno z kryteriów stopu. Pełne animacje, ukazujące proces zbieżności klatka po klatce, zostały dołączone do sprawozdania jako osobne pliki w formacie GIF.

8.1 Funkcja Schwefela

Na Rysunku 1 przedstawiono końcowe położenie cząstek dla funkcji Schwefela.



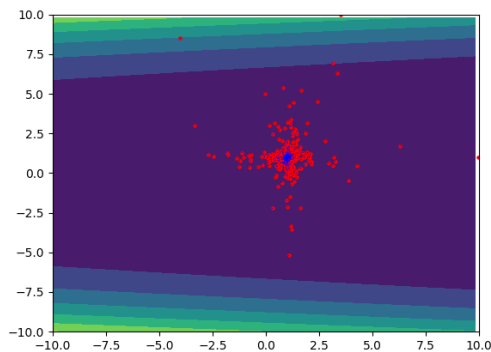
Rysunek 1: Końcowy stan roju dla funkcji Schwefela ($n = 2$).

Analiza wizualna:

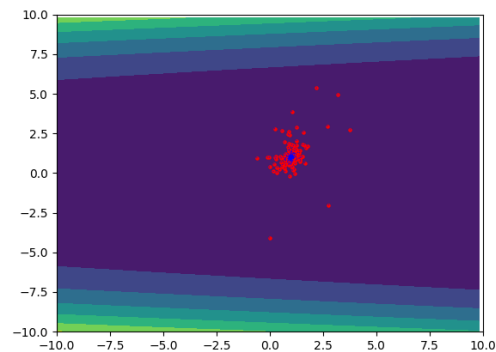
- **Obraz po lewej (Znane optimum):** Algorytm zbiegł do globalnego minimum, jednak widoczny jest pewien rozrzut cząstek wokół punktu optymalnego. Wynika to z natury kryterium stopu ($\|g_{best} - x_{opt}\| \leq 10^{-3}$) – symulacja jest przerywana natychmiast, gdy najlepsza cząstka (g_{best}) osiągnie zadaną bliskość celu. Pozostałe cząstki, które podążały za liderem, mogły nie zdążyć w pełni skupić się w tym samym punkcie przed zatrzymaniem algorytmu.
- **Obraz po prawej (Brak poprawy):** Widoczne jest **znacznie silniejsze skupienie** cząstek w punkcie minimum. Jest to efekt działania drugiego kryterium: algorytm musiał wykonać 50 dodatkowych iteracji, w których zmiana funkcji celu była znikoma ($\leq 10^{-6}$). Ten "czas stabilizacji" pozwolił całej populacji na dociągnięcie do pozycji lidera i utworzenie bardzo zwartej grupy.

8.2 Funkcja Rosenbrocka

Na Rysunku 2 przedstawiono wyniki dla funkcji Rosenbrocka.



(a) Kryterium 1: Znane optimum



(b) Kryterium 2: Brak poprawy

Rysunek 2: Końcowy stan roju dla funkcji Rosenbrocka ($n = 2$).

Analiza wizualna:

- **Obraz po lewej (Znane optimum):** Rój skutecznie zlokalizował obszar minimum w "dolinie Rosenbrocka" (punkt 1, 1). Podobnie jak w poprzednim zadaniu, natychmiastowe przerwanie pętli po osiągnięciu warunku odległości sprawia, że część cząstek jest jeszcze w fazie eksploracji lokalnego otoczenia optimum, tworząc luźniejszą chmurę.
- **Obraz po prawej (Brak poprawy):** Sytuacja wygląda analogicznie jak w poprzednim zadaniu. Cząstki tworzą tutaj znacznie gęstsze skupisko niż w przypadku pierwszego kryterium. Jest to efekt dodatkowych 50 iteracji, które algorytm musiał wykonać, by potwierdzić brak poprawy wyniku – ten czas pozwolił reszcie roju na dociągnięcie do lidera i zebranie się w jednym punkcie.